Milestone F

Updated C4 Container & Component Diagrams |
Remote API | Async Execution | Data Sharing

Embedded Systems - Face Detection Game Project

SDU - Software Engineering

May 2026

1. Updated C4 Container & Component Diagrams

Milestone F adds a web component (Flask REST API) running as a daemon thread inside the same Python process. The diagrams below show the new container boundary and how the web server interacts with the game controller.

1.1 C4 Container Diagram - PlantUML Source

```
@startuml Milestone_F_Container
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Container.puml
LAYOUT_WITH_LEGEND()
title C4 Container Diagram - Full System with Web Component (Milestone F)

Person(player, "Player", "Uses hardware buttons or keyboard to play")
Person(admin,  "Administrator", "Monitors game remotely via HTTP client")

System_Ext(camera, "Camera",       "USB webcam or Pi Camera Module")
System_Ext(display,"HDMI Display", "Renders game video output")
System_Ext(gpio,   "GPIO Hardware","Raspberry Pi buttons and LEDs")

System_Boundary(sys, "Embedded Face Detection Game (single Python process)") {
  Container(game, "Game Application", "Python / OpenCV / MediaPipe",
    "Main game loop on the OS main thread.\nCaptures frames, runs face detection, drives\n"
    "game state machine, controls GPIO LEDs, renders HUD.")
  Container(web, "Monitoring Web Server", "Python / Flask",
    "REST API on port 5000. Runs as a daemon thread\n"
    "inside the same process. Reads and controls game state\nvia registered Python callbacks.")
}

Rel(player, game,   "Interacts via",    "GPIO buttons / keyboard")
Rel(admin,  web,    "Queries & controls","HTTP / port 5000")
Rel(camera, game,   "Provides frames",  "cv2.VideoCapture")
Rel(game,   display,"Renders to",       "cv2.imshow / HDMI")
Rel(gpio,   game,   "Button events",    "GPIO daemon thread")
Rel(game,   gpio,   "LED outputs",      "GPIO BCM write")
Rel(game,   web,    "Shares live state","Python callbacks / shared object")
Rel(web,    game,   "Control commands", "Callback invocation on HTTP request")
@enduml
```

1.2 Container Diagram - ASCII Overview

```
[Player] ----GPIO/KB---->  +-----------------------------------------------+
[Admin]  ----HTTP:5000-->  |  Face Detection Game  (one Python process)    |
                           |                                               |
                           |  +-----------------------------------+        |
                           |  | Game Application (main thread)  |        |
                           |  |                                   |        |
                           |  |  - cv2.VideoCapture (camera)    |        |
                           |  |  - FaceDetector (MediaPipe/DNN) |        |
                           |  |  - CameraFaceDetector (state    |        |
                           |  |    machine, scoring, GPIO LED)  |        |
                           |  |  - HUD rendering -> HDMI        |        |
                           |  +----------------|-----------------+        |
                           |       shared object / callbacks               |
                           |  +----------------v-----------------+        |
```

```
                          |  | Monitoring Web Server (daemon)   |        |
                          |  |  Flask app, port 5000            |        |
                          |  |  GET  /game/status               |        |
                          |  |  POST /game/start  /game/pause   |        |
                          |  |  GET  /metrics   /health         |        |
                          |  +----------------------------------+        |
                          +-----------------------------------------------+
```

1.3 Component Diagram Update - PlantUML Source

```
@startuml Milestone_F_Component
!include https://raw.githubusercontent.com/plantuml-stdlib/C4-PlantUML/master/C4_Component.puml
LAYOUT_WITH_LEGEND()
title C4 Component Diagram - with Web Component (Milestone F)

Container_Boundary(game_ct, "Game Application (main thread)") {
  Component(cd,  "CameraFaceDetector", "Python Class",
    "Game controller. Exposes _get_status(), start_game(),\n"
    "toggle_pause(), _adjust_max_faces() as callbacks.")
  Component(fd,  "FaceDetector",       "Python Class",  "Face detection pipeline.")
  Component(hud, "HUD Renderer",       "Functions",     "Game overlay rendering.")
  Component(hw,  "HardwareController", "Python Class",  "GPIO buttons and LEDs.")
}

Container_Boundary(web_ct, "Monitoring Server (daemon thread)") {
  Component(ms,  "MonitoringServer",   "Python Class / Flask",
    "Wraps Flask app. Registers routes. Holds callback\n"
    "references to the game controller methods.")
  Component(routes, "Flask Routes",   "Flask decorators",
    "/health  /game/status  /game/start\n/game/pause  /game/max-faces  /metrics")
}

Rel(cd,     ms,     "Registers callbacks at startup")
Rel(ms,     routes, "Mounts routes on Flask app")
Rel(routes, cd,     "Invokes game callbacks on HTTP request")
Rel(cd,     fd,     "Face detection per frame")
Rel(cd,     hw,     "LED feedback + button events")
Rel(cd,     hud,    "HUD rendering")
@enduml
```

2. Remote API Description

2.1 Protocol and Network Details

Property	Value
Protocol	HTTP/1.1 (REST)
Host	0.0.0.0 (binds all network interfaces on the Raspberry Pi)
Port	5000 (configurable via MonitoringServer(port=...))
Data format	JSON (application/json)
HTTP methods	GET for read-only queries, POST for state-changing commands
Auth	None (local network assumed; add token middleware if exposed publicly)

2.2 Endpoints

Method	Endpoint	Description	Response
GET	/health	Server liveness check	{"status":"ok","timestamp":"..."}
GET	/game/status	Full game state snapshot	GameStatus JSON object
POST	/game/start	Start or restart the game	{"message":"Game started"}
POST	/game/pause	Toggle pause / resume	{"message":"Game paused/resumed"}
POST	/game/max-faces	Adjust face target (see body)	{"message":"Max faces adjusted..."}
GET	/metrics	FPS, face count, score, state	Metrics JSON object

2.3 Request / Response Examples

GET /game/status

```
GET http://192.168.1.42:5000/game/status

HTTP/1.1 200 OK
Content-Type: application/json

{
  "state":                "running",
  "score":                7,
  "strikes":              1,
  "max_faces":            2,
  "current_faces":        2,
  "fps":                  18.4,
  "switch_time_remaining": 1.23,
  "current_switch_time":  2.3,
  "game_over":            false,
  "timestamp":            "2026-05-04T14:32:01.456789",
  "backend":              "mediapipe"
}
```

POST /game/max-faces (increment by direction)

```
POST http://192.168.1.42:5000/game/max-faces
```

```
Content-Type: application/json

{ "direction": "up" }          # increment target by 1  (max 5)
# OR
{ "direction": "down" }        # decrement target by 1  (min 1)
# OR
{ "value": 3 }                 # set to specific value

HTTP/1.1 200 OK
{ "message": "Max faces adjusted to 3" }
```

3. Asynchronous Execution Mechanism

The web server and the game loop run concurrently inside a single Python process using the threading module. This avoids the complexity of multi-process IPC while keeping the HTTP server completely transparent to the game loop.

3.1 Threading Architecture

```
# Thread layout at runtime:
#
#  MainThread      -  game loop: camera capture, face detection,
#                     state machine, HUD render, keyboard input
#
#  Thread-Flask    -  Flask WSGI server (daemon=True)
#                     handles incoming HTTP requests
#
#  Thread-BtnStart -  GPIO pin 17 polling (daemon=True)
#  Thread-BtnPause -  GPIO pin 23 polling (daemon=True)
#  Thread-BtnLeft  -  GPIO pin 27 polling (daemon=True)
#  Thread-BtnRight -  GPIO pin 22 polling (daemon=True)
#
#  Thread-LED-*    -  short-lived daemon threads created by flash_led()
#                     for non-blocking LED flash sequences
```

3.2 Flask Daemon Thread - Code

```
# monitoring_server.py
def start(self) -> None:
    if not self.app:
        print("Flask not available. HTTP monitoring disabled.")
        return

    def run_server():
        # Flask's built-in Werkzeug WSGI server handles concurrency
        # with its own internal thread pool (threaded=True)
        self.app.run(
            host=self.host,     # "0.0.0.0" - all interfaces
            port=self.port,     # 5000
            debug=False,        # must be False in daemon thread
            threaded=True,      # each request gets its own thread
        )

    self.server_thread = threading.Thread(target=run_server, daemon=True)
    self.server_thread.start()
```

```
    # daemon=True means this thread is automatically killed when the
    # main thread (game loop) exits - no explicit shutdown needed
```

3.3 Why the Web Server Does Not Interfere With the Game Loop

- The Flask thread blocks inside app.run() waiting for HTTP connections. It consumes CPU only when an HTTP request arrives, which is infrequent (<1% CPU overhead).
- HTTP handlers execute in Flask worker threads (threaded=True), never on the main thread. The game loop is never blocked by an HTTP request.
- daemon=True ensures the Flask thread is cleaned up automatically when the game exits via the finally block - no orphaned processes.
- The Python GIL serialises bytecode execution between threads, so simple Python variable reads/writes are effectively atomic for the small integer and boolean types used as game state.

4. Data Sharing Between Web Server and Game

4.1 Mechanism: Python Callback References

Instead of shared memory, queues, or pipes, the MonitoringServer holds direct references to bound methods of the CameraFaceDetector instance. These callbacks are registered once at startup:

```
# In CameraFaceDetector.__init__():
if self.monitoring:
    # READ callbacks - web server calls these to get current state
    self.monitoring.set_game_state_callback(self._get_status)

    # WRITE callbacks - web server calls these to control the game
    self.monitoring.set_start_game_callback(self.start_game)
    self.monitoring.set_pause_game_callback(self.toggle_pause)
    self.monitoring.set_adjust_max_faces_callback(self._adjust_max_faces)

    self.monitoring.start()  # launch daemon thread
```

4.2 Read Path: Status Snapshot

```
# Called by Flask route on GET /game/status
# Executes in a Flask worker thread
def _get_status(self) -> GameStatus:
    fps = (sum(self._fps_samples) / len(self._fps_samples)
           if self._fps_samples else 0.0)
    # Reads simple Python attributes - safe under GIL without a lock
    return GameStatus(
        state=self.state.value,          # str (Enum.value)
        score=self.score,                # int
        strikes=self.strikes,            # int
        max_faces=self.max_faces,        # int
        current_faces=self.current_faces,# int
        fps=fps,                         # float
        switch_time_remaining=max(0.0,
            self.get_switch_time() - self.get_switch_elapsed_time()),
        current_switch_time=self.get_switch_time(),
        game_over=self.game_over,        # bool
```

```
        timestamp=datetime.now().isoformat(),
        backend=self.detector.backend,   # str
    )
```

4.3 Write Path: Control Commands

POST endpoints invoke the same transition methods (start_game, toggle_pause, _adjust_max_faces) that hardware buttons use. Because these methods only modify simple Python integer/bool/enum attributes, and Python's GIL prevents two threads from executing Python bytecode simultaneously, no explicit locking is required for the attribute writes.

```
# Flask route handler (runs in Flask worker thread):
@self.app.route('/game/start', methods=['POST'])
def start_game():
    if self._start_game_callback:
        self._start_game_callback()   # -> CameraFaceDetector.start_game()
        return jsonify({"message": "Game started"}), 200

# The callback modifies game state:
def start_game(self) -> None:        # runs in Flask worker thread!
    self.state   = GameState.RUNNING  # atomic under GIL
    self.score   = 0                  # atomic under GIL
    self.strikes = 0                  # atomic under GIL
    ...
```

4.4 Thread Safety Summary

Concern	How it is handled
Read of int/bool/str from Flask thread	Safe: Python GIL ensures single-bytecode atomicity
Write of int/bool/str from Flask thread	Safe: single assignment is one bytecode (STORE_ATTR)
Write during active game-loop read	Rare race; worst case: one stale frame of game state, no corruption
fps deque (collections.deque)	deque.append() is thread-safe in CPython (deque uses a C-level lock)
LED flash threads	HardwareController spawns daemon threads; each thread owns its own pin write